

3주 차 보고서

(2026년 1 월 11일 ~ 2026년 1 월 17일)

작성자

박신우

이번 주 회의 내용

- 그동안 모아온 에셋 후보 중에서트럭, 좀비, 맵 에셋을 우선적으로 확정 선정함
- 기타 에셋은 추후 작업 진행 상황을 보면서 후보 내에서 선별 또는 추가 선정하기로 결정
- 에셋 전용 레포지토리 생성

이번 주 한 일

1.12 코드 수정

OnZombieDamaged 델리게이트에 FHitResult(피격 위치 정보) 인자를 추가하여, 이펙트가 좀비 위치가 아니라 실제 피격 위치에 생성되도록 로직 개선을 개선하였다.

1.13 트럭 ,회의 진행 , 에셋 이주

회의에서 결정해서 산 에셋들이 버전이 안맞아서 에셋 migrate 작업을 진행했다



트럭에 chaos vehicle 시스템을 적용하려는데,피직스 에셋을 잘못 설정해서 그런지 트럭이 이상하게 굴러다니는 이슈가 생겼다.

왜 안 되나 했더니 Wheel setup index에 한쪽 바퀴의 스켈레톤 이름을 내가 잘 못 적어서 생긴 문제였다. 시스템에서 바퀴가 Bone에 붙지 않았다고 생각해서 저렇게 된 것 같다. 문제를 해결하여 트럭이 정상적으로 서 있게 되었다.



이번 주 한 일

1.14 트럭 인풋맵핑 , 트럭 애니메이션

플레이어가 트럭을 조종할 수 있도록 Throttle/Steer/Brake 입력을 바인딩 해주었다. 트럭에 애니메이션을 적용하여 좌회전,우회전할 때 바퀴가 회전하고, 차량이 움직일 때 바퀴가 굴러가도록 하였다.



1.16 인터페이스, 상호작용 트리거 컴포넌트

인터페이스 기반 상호작용 (IInteractInterface):

트럭, 아이템, 문 등 다양한 액터들이 공통적으로 가질 수 있는 Interact 함수를 정의했다.

상호작용 트리거 컴포넌트 (UInteractTriggerComponent):

USphereComponent를 상속받아 액터 주변의 상호작용이 가능한 범위를 설정해주었다.

플레이어 진입 시 캐릭터에게 상호작용 대상을 등록하고, 이탈 시 대상을 해제하는 상태 관리 로직을 구현했다.

1.17 캐릭터 컨트롤러 빙의 , 보고서 정리

```
void ATruck::Interact_Implementation(AFPSBaseCharacter* Character)
{
    if (!Character) return;

    // 캐릭터를 조종하던 컨트롤러를 가져옴
    AController* PlayerController = Character->GetController();

    if (PlayerController)
    {
        //빙의대상을 캐릭터에서 트럭으로 변경
        PlayerController->Possess(this);

        UE_LOG(LogTemp, Log, TEXT("Truck Possessed!"));
    }
}
```

PlayerController->Possess()를 통해 캐릭터에서 트럭으로 조작 권한이 넘어가도록 하였다. 플레이어가 트럭 가까이에서 f키를 누르면 트럭을 조종할 수 있도록 하였다.

다음 주 할 일
언리얼 엔진에서 인터페이스 헤더 파일에서 함수 선언 방식 공부 캐릭터 1스테이지 인벤토리 기능 추가 트럭 운전 사운드 추가

특이사항

3주 차 보고서

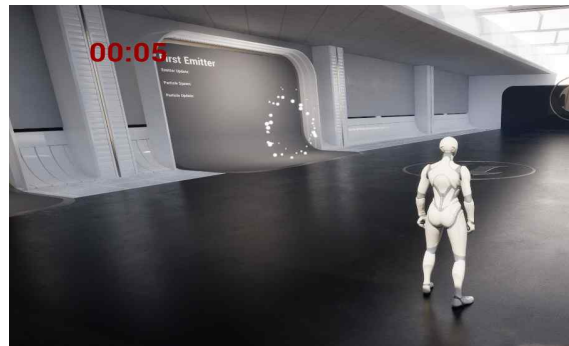
(2026년 1 월 11 일 ~ 2026년 1 월 17 일)

작성자

안윤진

이번 주 한 일

- < BaseUI > c++ class 제작
5분 타이머 설정. 시간에 따라 변화하는 타이머 UI구현
- <TimerUI> 추가 - < BaseUI > 기반 위젯 블루프린트
(이 부분은 C++이 아닌 UMG로 편집해야하기에 블프로 제작)
타이머 UI 애니메이션 2종 구현 (미적 감각 이슈로 후에 다시 수정 예정)
-20초부터 타이머 흔들리는 애니메이션
-5초부터 빨간색과 팝업 느낌의 애니메이션
- ++ [수정 예정]
후에 타이머에 맞게 음향 추가
타이머 종료시 삭제 모션 추가



이번 주 한 일

PCG를 통한 맵 제작을 위해 PCG 플러그인 추가

PCG는 CPU 기반으로 오브젝트 배치를 생성 -> Static Mesh로 렌더링 되기에 대규모 환경 구성 시 성능과 작업 효율 면에서 장점이있다 판단해 활용하기로 선택했다.

- <PCG_House> PCG Graph 추가

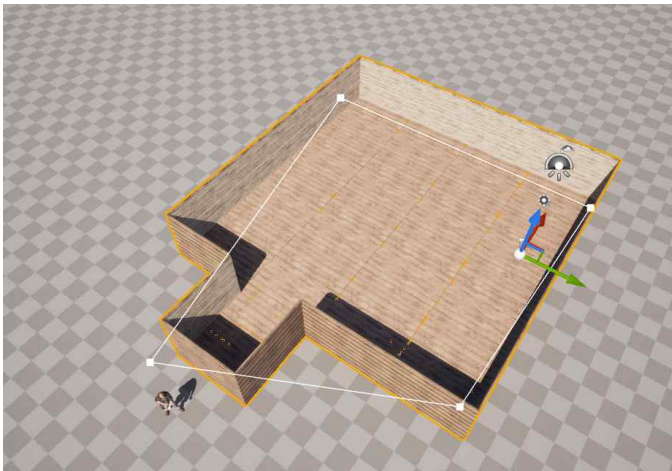
bp에서 변수로 선택된 메쉬를 이용해 렌더링

이때 spawn point의 방향과 회전을 수정해 바닥을 감싸는 벽 형태 제작

- <BP_House1> 블프 추가

스플라인과 PCG_House를 기반으로 바닥 메쉬와 벽 메쉬 배치

점 조절에 따른 집 바닥과 벽 형태 자동 변경



- 계획

첫 번째 파밍 맵 제작에서 집들을 각 pcg를 통한 액터로 저장.

그 액터들 배치를 pcg를 통한 랜덤 배치로 각 플레이 판마다 변하는 맵 구현

- 현실

pcg를 통한 집 제작에서 큰 집이 아니기에 직접 메쉬 배치로 집 제작이 더 빠를 것 같음

언리얼이 계속 터짐

디테일한 집 디자인이 어려움 (창문이 달린 벽으로 교체 등)

다음 주 할 일

첫 번째 맵 제작 - 랜드 스케이프 조정, Pcg를 통한 집들 랜덤 배치.
좀비 애니메이션 제작 도전

특이사항

잘 진행되다 어느 순간부터 스플라인의 점을 옮기거나 다른걸 할 때 엔진 작동이 멈추고 돌아오지 않음. 그런데 한글 등 다른 앱을 실행시켜도 가끔씩 터짐. 컴퓨터 문제인지...
PCG에 따른 과부하 같은데, 정확한 원인 파악과 해결 필요,

3주 차 보고서

(2025년 1 월 11 일 ~ 2026년 1 월 17 일)

작성자

배주환

이번 주 회의 내용

이번 주 한 일

이번주에는 몸이 안좋아서 별로 공부를 못했다.

캐시, 메모리 모델, cpu 파이프라인에 대해서 공부를 했는데 컴퓨터 구조와 관련된 내용이었다.

캐시라는 중간 저장소가 있는데 cpu가 core가 여러개가 있고 L1캐시, L2캐시 같이 임시저장소같은 것이 있는 걸 볼 수 있다. L1캐시 같은 경우는 용량이 좀 작지만 코어와 가까이 있고, L2캐시는 용량은 좀 더 크지만 코어와 좀 더 멀리 있다. 여기까지 체크했는데 없으면 RAM까지 가서 체크한다.

캐시에는 두가지의 철학이 있는데

첫번째로 Temporal Locality

시간적으로 보면, 방금 주문한 테이블에서 추가 주문이 나올 확률이 높다는 것이다.

두번째로 Spatial Locality

공간적으로 보면, 방금 주문한 사람 근처에 있는 사람이 추가 주문을 할 확률이 높다.

싱글쓰레드를 사용할때는 컴파일러의 개입이 문제가 없었지만 멀티쓰레드를 사용한다면 컴파일러의 개입 때문에 원하는대로 출력이 안될 수도 있다. 그래서 가시성 및 코드 재배치가 중요하다.

캐시와 파이프라인에 대해서 빨래에 대해서 비유를 해보자.

세탁, 건조, 다림질, 빨래날기의 4단계가 있다고 할때 빨래바구니가 여러개 있을때 한 바구니가 끝날때까지 다른 바구니들이 쉬고 있으면 낭비가 심하다. 그래서 현실적으로는 한 단계가 끝나면 다음 바구니를 시작하고 다음 단계로 가는 방법이 효율적이라는 것을 알 수 있다.

하지만 빨래바구니 별로 크기와 중요도가 달라서 각각 단계를 해결하는데 걸리는 시간이 다르다고 가정을 하면 곧이 곧대로 받은 순서대로 실행하기보다는 순서를 효율적인 방식으로 바꾸는 것이 좋을 것이다.

CPU가 이런 방식으로 동작한다고 보면 된다.

atomic 연산에 한해, 모든 쓰레드가 동일 객체에 대해서 동일한 수정 순서를 관찰할 수 있다.

TLS에 대해서도 공부를 했다.

TLS(Thread Local Storage)란 무엇이나면 큰 영역에 해당하는 Heap영역이나 데이터 영역에 매번 갈 필요 없이 TLS에 넣어놓고 사용할 수 있다.

TLS는 스레드마다 사용할 수 있는 전역메모리같은 느낌이다.

LockStack과 LockQueue를 만들어서 동기화 스택과 큐를 만들어봤다.

```
atomic<bool> flag;

int main()
{
    flag = false;

    // flag = true;
    flag.store(true, memory_order::memory_order_seq_cst);

    // bool val = flag;
    bool val = flag.load(memory_order::memory_order_seq_cst);

    // 이전 flag 값을 prev에 넣고, flag 값을 수정
    {
        bool prev = flag.exchange(true);
        //bool prev = flag;
        //flag = true;
    }

    // CAS(Compare-And-Swap) 조건부 수정
    {
        bool expected = false;
        bool desired = true;
        flag.compare_exchange_strong(expected, desired);
    }
}

#include <mutex>

template<typename T>
class LockStack
{
public:
    LockStack() {}

    LockStack(const LockStack&) = delete;
    LockStack& operator=(const LockStack&) = delete;

    void Push(T value)
    {
        lock_guard<mutex> lock(_mutex);
        _stack.push(std::move(value));
        _condVar.notify_one();
    }

    bool TryPop(T& value)
    {
        lock_guard<mutex> lock(_mutex);
        if (_stack.empty())
            return false;

        // empty -> top -> pop
        value = std::move(_stack.top());
        _stack.pop();
        return true;
    }

    void WaitPop(T& value)
    {
        unique_lock<mutex> lock(_mutex);
        _condVar.wait(lock, [this] { return !_stack.empty(); });
        value = std::move(_stack.top());
        _stack.pop();
    }

private:
    stack<T> _stack;
    mutex _mutex;
    condition_variable _condVar;
};
```

```

thread_local int32 LThreadId = 0;

void ThreadMain(int32 threadId)
{
    LThreadId = threadId;

    while (true)
    {
        cout << "Hi! I am Thread" << LThreadId << endl;
        this_thread::sleep_for(1s);
    }
}

int main()
{
    vector<thread> threads;

    for (int32 i = 0; i < 10; i++)
    {
        int32 threadId = i + 1;
        threads.push_back(thread(ThreadMain, threadId));
    }

    for (thread& t : threads)
        t.join();
}

```

```

atomic<bool> ready;
int32 value;

void Producer()
{
    value = 10;

    ready.store(true, memory_order::memory_order_seq_cst);
}

void Consumer()
{
    while (ready.load(memory_order::memory_order_seq_cst) == false)
        ;

    cout << value << endl;
}

int main()
{
    ready = false;
    value = 0;
    thread t1(Producer);
    thread t2(Consumer);
    t1.join();
    t2.join();

    // Memory Model(정책)
    // 1) Sequentially Consistent (seq_cst)
    // 2) Acquire-Release (acquire, release)
    // 3) Relaxed (relaxed)

    // 1) seq_cst (가장 엄격 = 컴파일러 최적화 어지 적음 = 직관적)
    // 가시성 문제 바로 해결! 코드 재배치 바로 해결!

    // 2) acquire-release
    // 딱 중간!
    // release 명령 이전의 메모리 명령들이, 해당 명령 이후로 재배치 되는 것을 금지
    // 그리고 acquire도 같은 변수를 읽는 쓰레드가 있다면
    // release 이전의 명령들이 -> acquire 하는 순간에 관찰 가능(가시성 보장)

    // 3) relaxed (자유롭다 = 컴파일러 최적화 어지 많음 = 직관적이지 않음)
    // 너무나도 자유롭다
    // 코드 재배치도 멋대로 가능! 가시성 해결은 No!
    // 가장 기본 조건(동일 객체에 대한 동일 관전 순서만 보장)

    // 인텔, AMD의 경우 애당초 순차적으로 일관성을 보장을 해서,
    // seq_cst를 써도 별다른 부하가 없음
    // ARM의 경우 꽤 차이가 있다
}

```

다음 주 할 일

Lock free에 대한 공부를 하고 ThreadManager라는 프레임워크를 만들어볼것이다. 다음 주에는 멀티쓰레드에 대한 공부 끝날 예정이다.

특이사항

